Implementing FACE on your own data

Elizabeth Bersson

This vignette shows how to implement the FACE model described in *Feature aware covariance estimation, with application to mixtures of chemical exposures*. It walks through a complete analysis on a synthetic chemical exposure dataset while highlighting how to substitute in your own data.

The code here is drawn from `SYNTHETIC_APPLICATION.R`, which runs the same analysis end to end as a plain script. Refer to it if you would rather read the entire workflow in one place.

Contents

When to use FACE

FACE estimates a `p x p` covariance matrix from `n` observations based on *feature covariates*, covariates that vary with the outcome index, and describe auxiliary information about the outcome variables themselves. In a chemical exposure data set, for example, feature covariates might include chemical class, assay time, or chemical-molecular properties like vapor pressure.

The sampling model is a Bayesian factor model,

$$y_i = \Lambda \eta_i + \epsilon_i, \qquad \epsilon_i \sim N_p(0, D), \qquad \eta_i \sim N_k(0, I),$$

with a prior that regresses the loadings on the features:

$$\lambda_j \sim N_k(\Gamma^\top x_j,\ \tau^2 d_j \Theta).$$

Here x_j is the vector of features for variable j. Variables with similar features are shrunk toward similar loadings, and so toward similar covariance structure.

You need two objects:

object	dimension	meaning
`Y`	`n x p`	your data: n observations, p response variables
`X`	`p x q`	your features: q features for each of the p response variables

Ensure your data matrices are correctly aligned! Row `j` of `X` must describe column `j` of `Y`.

Setup

```
source("helpers.R")          # helper functions
source("functions.R")        # FACE specific functions

library(parallel)            # used within FACE Gibbs sampler
library(dplyr)               # used for plotting
library(RColorBrewer)        # used for plotting

set.seed(123)                # set a seed for reproducibility
```

Step 1: prepare the data matrix `Y`

Our synthetic example mimics the article's motivating study: it consists of exposures to 21 chemicals measured by 2 different sampling methods on `n = 73` participants. This yields `p = 42` response variables for each participant.

```
# import response data
Y = as.matrix(read.csv("synethetic_data.csv"))
p = ncol(Y)
n = nrow(Y)

# preview the data
Y[1:5,1:6]
#>              V1         V2          V3         V4         V5          V6
#> [1,] -0.64536380 -0.9238735 -1.56656957 -1.6708523 -0.5567200 -1.15430463
#> [2,] -0.29400873 -0.6238325 -0.04464788 -0.6340139 -0.7084628 -0.22598183
#> [3,]  1.60892070  2.0404311  2.78188571  2.2296279  2.9947795  1.84644197
#> [4,]  0.02584624 -0.1011214 -0.80643314  0.2913706 -0.1402906  0.35085526
#> [5,]  0.08837284 -0.6585577  0.16802247  0.2230546  0.8899882 -0.07232312
dim(Y)          # n x p
#> [1] 73 42
```

For your data. `Y` is an `n x p` numeric matrix.

- **Center it.** The model has no intercept, so each column must be mean zero.

- **Consider scaling.** `scale()` puts every variable on a common scale, which makes the resulting estimate a correlation-scale object and keeps the prior on `D` sensible. For exposure concentrations, log-transform before scaling.
- **Column order matters** — fix it now, because `X` must follow it. Here the order is chemical-within-method: chemicals 1–21 for wristbands, then chemicals 1–21 for dust.

Step 2: build the feature matrix `X`

Our data has a corresponding data dictionary that we will use to construct an outcome feature matrix `X` which will consist of one row per variable in `Y`. We will use these auxilary variables to construct meta covariates for our analysis.

```
# import data dictionary
data.dictionary = read.csv("data_dictionary.csv")

# print data
head(data.dictionary)
#>   Variable Chemical Chemical.Class Measurement.Source Vapor.Pressure
#> 1       V1    EHDPP            OPE                 WB   0.0001261226
#> 2       V2     TCEP            OPE                 WB   0.0861260100
#> 3       V3     TCPP            OPE                 WB   0.0394633100
#> 4       V4    TDCPP            OPE                 WB   0.0006999405
#> 5       V5      TPP            OPE                 WB   0.0003666355
#> 6       V6  2IPPDPP            OPE                 WB   0.0000466627
#>   Production.Volume
#> 1               yes
#> 2               yes
#> 3               yes
#> 4               yes
#> 5               yes
#> 6                no
```

Categorical features

`getCatDesignMat()` turns a length-`p` vector of group labels into a binary indicator matrix. Chemical class is the natural example.

```
# construct the feature covariate
class.mat = getCatDesignMat(data.dictionary$Chemical.Class)

dim(class.mat)
#> [1] 42  3
colnames(class.mat)
#> [1] "OPE"       "phthalate" "phenol"
```

Matrix-variate structure

When each observation is naturally a `p1 x p2` matrix flattened into a length-`p` vector — here chemical-by-method — `getMatDesignMat(p1, p2)` returns indicators for row membership and column membership. This lets FACE learn a chemical effect and a method effect separately.

```
# extract matrix dimensions from auxilary data
p1 = length(unique(data.dictionary$Chemical))
p2 = length(unique(data.dictionary$Measurement.Source))
```

```r
# construct the feature covariate
struct.mat = getMatDesignMat(p1, p2)

dim(struct.mat)
#> [1] 42 23
colnames(struct.mat)
#>  [1] "R1"  "R2"  "R3"  "R4"  "R5"  "R6"  "R7"  "R8"  "R9"  "R10" "R11" "R12"
#> [13] "R13" "R14" "R15" "R16" "R17" "R18" "R19" "R20" "R21" "C1"  "C2"
```

Note the vectorization convention: `getMatDesignMat` assumes `Y`'s columns were formed by `c()` on the `p1 x p2` matrix, i.e. all `p1` chemicals for measurement source 1, then all `p1` chemicals for measurement source 2.

Continuous features

Any numeric property of a variable can be a column of `X`. Here we extract vapor pressure and an indicator for high production volume from the data dictionary.

```r
# extract the continuous variables we are interested in
cont.covs = data.dictionary[,c("Vapor.Pressure","Production.Volume")]
# convert a ("yes","no") variable to numeric binary
cont.covs$Production.Volume = as.numeric(as.factor(cont.covs$Production.Volume))-1
```

Put them together

```r
X = cbind(struct.mat,class.mat,cont.covs) |>
  as.matrix()

dim(X)          # p x q
#> [1] 42 28
colnames(X)
#>  [1] "R1"               "R2"               "R3"
#>  [4] "R4"               "R5"               "R6"
#>  [7] "R7"               "R8"               "R9"
#> [10] "R10"              "R11"              "R12"
#> [13] "R13"              "R14"              "R15"
#> [16] "R16"              "R17"              "R18"
#> [19] "R19"              "R20"              "R21"
#> [22] "C1"               "C2"               "OPE"
#> [25] "phthalate"        "phenol"           "Vapor.Pressure"
#> [28] "Production.Volume"
```

If you have no features at all, pass `X = NA`. FACE then uses an intercept column, giving a factor model that shrinks all loadings toward a common value.

Step 3: group the features with `which.cov.group`

FACE uses a generalized ridge prior on the rows of Γ, so that each *set* of related feature columns gets its own shrinkage scale. This matters when your features are on different scales or of different types.

`which.cov.group` is an integer vector of length `q` labeling which group each **column of `X`** belongs to. Columns sharing a label share a variance parameter, which is estimated from the data.

```r
which.cov.group = c(rep(1, p1),                        # chemical indicators
                    rep(2, p2),                        # measurement source indicators
                    rep(3, length(unique(chemclass))), # chemical class
```

```
                    4,                                  # production volume
                    5)                                  # vapor pressure

length(which.cov.group) == ncol(X)
#> [1] TRUE
```

Rule of thumb: one group per conceptual block of features. Give each standalone continuous covariate its own group. Omit the argument entirely (leave it `NA`) to use a common scale for all features.

Step 4: run the sampler

```
model.out = FACE_cusp_GS(Y, X,
                         which.cov.group = which.cov.group,
                         k = p,                         # upper bound on # factors
                         S = 2000,                      # total MCMC iterations
                         burnin = 1000,                 # number of MCMC burnin iterations
                         my.seed = 1)                   # seed for replicability
```

The arguments worth understanding:

input	description
`k`	An **upper bound** on the number of latent factors
`S`, `burnin`, `thin`	MCMC parameters
`my.seed`	seed for reproducibilty
`save.all`	what to output: `0` (default) returns thinned post-burn-in draws for key parameters. `1` returns every iteration. `"ICO"` returns only the running mean of the inverse covariance — use if `p` is very large.

Key function outputs: `cov` are MCMC samples of the posterior covariance matrix and `cov.inv` MCMC samples of the posterior precision matrix, both are stored as vectorized `p x p` matrices (one iteration per row).

Step 5: summarize the posterior

Point estimation

The paper reports the Bayes estimate under Stein's loss, which is the inverse of the posterior mean of the precision matrix:

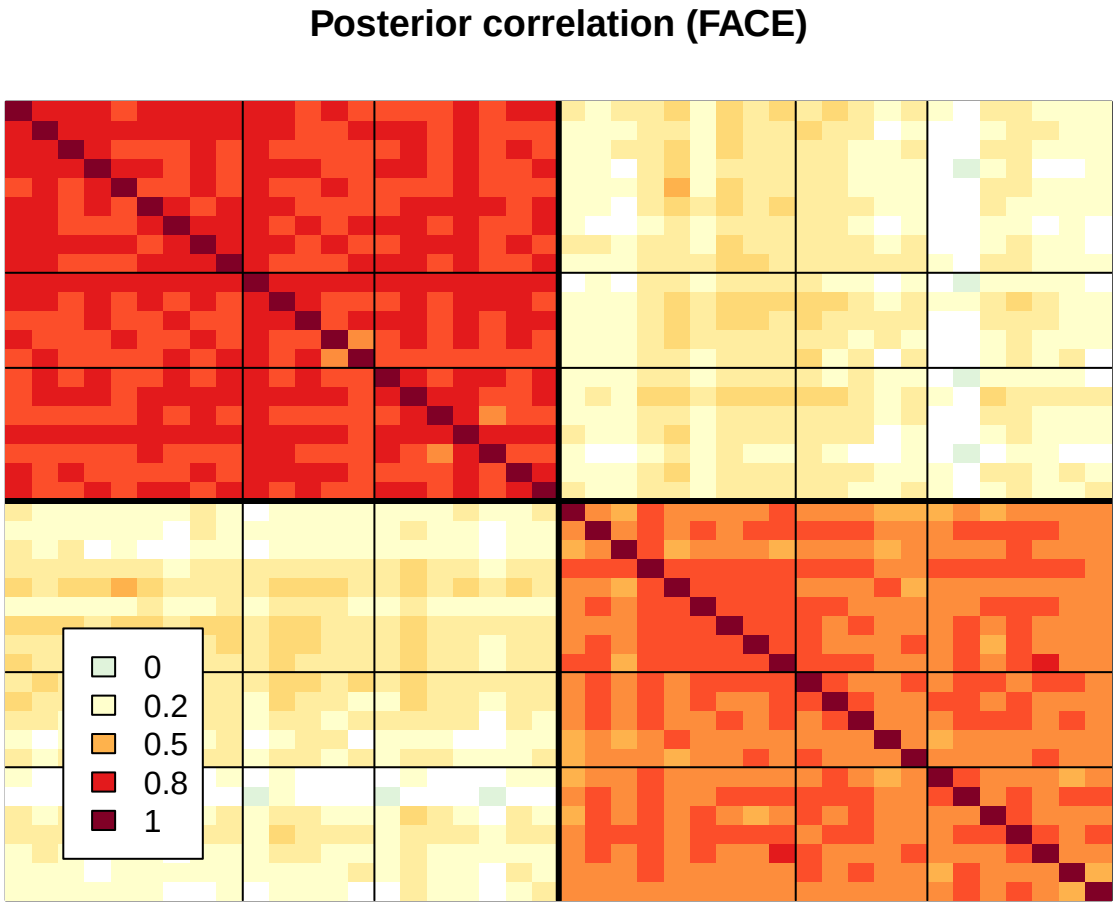
```
face.cov = qr.solve(matrix(colMeans(model.out$cov.inv), ncol = p))
face.cor = cov2cor(face.cov)
```

Plotting

```
col.pal = c(brewer.pal(9, "GnBu")[2], "#FFFFFF", brewer.pal(9, "YlOrRd"))
col.lims = c(min(0, min(face.cor)), 1)

image(x = 1:p, y = 1:p, z = face.cor[, p:1],
      col = col.pal, zlim = col.lims,
      xaxt = "n", yaxt = "n", xlab = "", ylab = "",
```

```
      main = "Posterior correlation (FACE)")
# block boundary between the two sampling methods
abline(h = p/2 + .5, v = p/2 + .5, col = "black", lwd = 3)
# chemical class boundaries
for (b in cumsum(table(chemclass))[1:2]) {
  abline(v = c(b, p/2 + b) + .5)
  abline(h = p - c(b, p/2 + b) + .5)
}
legend("bottomleft", inset = 0.052, bg = "white",
       legend = round(seq(col.lims[1], col.lims[2],
                          length.out = length(col.pal)), 2)[c(1,3,6,9,11)],
       fill = col.pal[c(1,3,6,9,11)], cex = 1)
```



The two diagonal blocks — strong correlation within each of the two measurement sources, moderate correlation between the measurement sources — reflect the structure expected for the synthetic dataset. `SYNTHETIC_APPLICATION.R` has a fuller version of this plot with variable class labels on the axes.

Template for your data

```
source("helpers.R")
source("functions.R")
library(parallel)

## 1. Y: n x p, centered (and usually scaled). Fix the column order here.
```

```r
Y = scale(your.data.matrix)
n = nrow(Y); p = ncol(Y)

## 2. X: p x q, one ROW per column of Y, in the same order.
X = your.covariate.matrix
stopifnot(nrow(X) == p)

## 3. which.cov.group: length q, one label per COLUMN of X, or NA.
which.cov.group = NA

## 4. Fit.
out = FACE_cusp_GS(Y, X,
                   which.cov.group = which.cov.group,
                   k = p,
                   S = 20000, burnin = 10000, thin = 1,
                   my.seed = 123)

## 5. Summarize.
cov.hat = qr.solve(matrix(colMeans(out$cov.inv), ncol = p))
cor.hat = cov2cor(cov.hat)
```